

Leverage-Driven Software Architecture: A Probabilistic Framework for Architectural Decision-Making

Anonymous Author
Anonymous Institution
anonymous@example.com

April 15, 2026

Abstract

We study leverage as a structural metric for software architecture. We model an architecture by its degrees of freedom (DOF), capability set, and modification surface, and define leverage as the ratio of capabilities to independent change points. In this setting we prove a software-engineering theorem arc: architectures with lower DOF have lower exact error probability under the derived reliability model, lower expected modification cost, and better leverage in fixed capability classes.

The resulting framework yields an actionable decision rule: among architectures meeting the same requirements, prefer the one with maximal leverage. We show that common software patterns fit this rule uniformly. Single-source-of-truth designs, nominal contracts, generic abstractions, convention-over-configuration, and normalization all realize the same structural mechanism: consolidating independently drifting definition loci while preserving capabilities.

We instantiate the framework on representative architectural choices and on an OpenHCS refactoring case, where scattered runtime checks collapse to a single nominal contract. The paper is accompanied by a Lean 4 artifact formalizing the leverage core, the probability model, and the main ordering theorems. This TOSEM-oriented version isolates the software-engineering contribution from the mathematical-physics results developed in the companion split.

1 Introduction

Software architecture decisions often trade flexibility, implementation effort, and error surface against one another, but the comparison is usually qualitative. This paper develops a quantitative framework for that comparison. The central idea is to treat an architecture’s independently drifting definition loci as its *degrees of freedom* (DOF), and to measure architectural leverage by the number of capabilities obtained per independent change point.

The software-engineering thesis is simple: for architectures with comparable capabilities, lower DOF yields lower modification burden and lower error surface. The framework turns that thesis into formal statements. We derive an exact error model from axis orthogonality, show that architectures with n DOF behave like series reliability systems with n failure points, and prove that maximizing leverage minimizes error probability in fixed capability classes. The same framework gives quantitative predictions for modification cost and explains why familiar consolidation patterns recur across software design.

This TOSEM-oriented split isolates the software-architecture contribution from the mathematical-physics material in the companion paper-3 split. The present paper keeps the architectural definitions, probability model, leverage theorems, software instances, and OpenHCS

demonstration. It removes the thermodynamic-selection framing as the main narrative and treats cross-paper physical results, where mentioned at all, only as background motivation.

1.1 Contributions

1. **Architectural leverage model.** We formalize architecture, DOF, capabilities, leverage, dominance, and modification complexity in a uniform framework for comparing software designs.
2. **Reliability-style error semantics for DOF.** We derive an exact error model from axis orthogonality and show that an architecture with n DOF is isomorphic to a series reliability system with n independent failure points.
3. **Leverage-based decision rule.** We prove that, at fixed capabilities, higher leverage implies lower error probability and lower expected modification burden.
4. **Cross-pattern explanatory power.** We show that SSOT, nominal contracts, generic endpoints, convention over configuration, and normalization instantiate the same consolidation mechanism.
5. **Mechanized core and case-oriented framing.** The leverage core is machine-checked in Lean 4, and the paper includes an OpenHCS example illustrating how DOF collapse appears in a real refactoring.

1.2 Scope

The paper is about software architecture and architectural decision-making. Performance, security, and other quality attributes are not reduced to leverage; rather, leverage quantifies one structural dimension of error exposure and maintenance cost. Likewise, the OpenHCS material is used as a mechanism-level demonstration, not as a broad empirical claim about all codebases.

1.3 Organization

The paper proceeds as follows. The next section defines the architectural model. The following section derives the error and reliability results. We then present the main leverage theorems, analyze recurring software patterns as leverage instances, give the OpenHCS demonstration, situate the work in software architecture and software engineering, and conclude.

2 Foundations

We formalize the core concepts: architecture state spaces, degrees of freedom, capabilities, and leverage.

2.1 Architecture State Space

Definition 2.1 (Architecture). An *architecture* is a tuple $A = (C, S, T, R)$ where:

- C is a finite set of *components* (modules, services, endpoints, etc.)
- $S = \prod_{c \in C} S_c$ is the *state space* (product of component state spaces)

- $T : S \rightarrow \mathcal{P}(S)$ defines valid *transitions* (state changes)
- R is a set of *requirements* the architecture must satisfy

Intuition: An architecture consists of components, each with a state space. The total state space is the product of component spaces. Transitions define how the system can evolve.

Example 2.2 (Microservices Architecture). • $C = \{U, O, P\}$ where $U = \text{UserService}$, $O = \text{OrderService}$, $P = \text{PaymentService}$

- $S_{\text{UserService}} = \text{UserDB} \times \text{Endpoints} \times \text{Config}$
- Similar for other services
- $S = S_{\text{UserService}} \times S_{\text{OrderService}} \times S_{\text{PaymentService}}$

2.2 Degrees of Freedom

Definition 2.3 (Degrees of Freedom). The *degrees of freedom* of architecture $A = (C, S, T, R)$ is a natural number $\text{DOF}(A) \in \mathbb{N}$ counting the independent axes of variation in the state space. Formally, it is the cardinality of a minimal complete orthogonal axis set for S (as established in `AbstractClassSystem`). In the Lean formalization, `Architecture.dof` is a field of type $\mathbb{N}$.

Operational meaning: DOF counts independent modification points. If $\text{DOF}(A) = n$, then n independent changes can be made to the architecture.

Proposition 2.4 (DOF Additivity). *For disjoint architectures $A_1 = (C_1, S_1, T_1, R_1)$ and $A_2 = (C_2, S_2, T_2, R_2)$ with $C_1 \cap C_2 = \emptyset$:*

$$\text{DOF}(A_1 \oplus A_2) = \text{DOF}(A_1) + \text{DOF}(A_2)$$

where $A_1 \oplus A_2 = (C_1 \cup C_2, S_1 \times S_2, T_1 \times T_2, R_1 \cup R_2)$.

¹

Proof. Components are disjoint ($C_1 \cap C_2 = \emptyset$), so the axis sets of A_1 and A_2 are disjoint. The axis set of $A_1 \oplus A_2$ is their disjoint union, giving $\text{DOF}(A_1 \oplus A_2) = |\text{axes}(A_1)| + |\text{axes}(A_2)| = \text{DOF}(A_1) + \text{DOF}(A_2)$. ■

Example 2.5 (DOF Calculations). 1. **Monolith:** Single deployment unit $\rightarrow \text{DOF} = 1$

2. **n Microservices:** n independent services $\rightarrow \text{DOF} = n$
3. **Copied Code:** Code duplicated to n locations $\rightarrow \text{DOF} = n$ (each copy independent)
4. **SSOT:** Single source, n derived uses $\rightarrow \text{DOF} = 1$ (only source is independent)
5. **k API Endpoints:** k independent definitions $\rightarrow \text{DOF} = k$
6. **m Config Parameters:** m independent settings $\rightarrow \text{DOF} = m$

¹ Lean: [L17](#), [L19](#)

2.3 Capabilities

Definition 2.6 (Capability Set). The *capability set* of architecture A is:

$$\text{Cap}(A) = \{r \in R \mid A \text{ satisfies } r\}$$

Examples of capabilities:

- “Support horizontal scaling”
- “Provide type provenance”
- “Enable independent deployment”
- “Satisfy single source of truth for class definitions”
- “Allow polyglot persistence”

Definition 2.7 (Capability Satisfaction). Architecture A *satisfies* requirement r (written $A \models r$) if there exists an execution trace in (S, T) that meets r ’s specification.

2.4 Leverage

Definition 2.8 (Leverage). The *leverage* of architecture A is:

$$L(A) = \frac{|\text{Cap}(A)|}{\text{DOF}(A)}$$

Special cases:

1. **Unbounded Leverage:** As $|\text{Cap}(A)|$ grows for fixed $\text{DOF}(A) = 1$, leverage grows without bound. This is the metaprogramming regime: a single source with an unbounded number of derived capabilities. In any fixed finite model, L is a positive rational; “ $L = \infty$ ” is a shorthand for this limit.
2. **Unit Leverage** ($L = 1$): Linear relationship (n capabilities from n DOF)
3. **Sublinear Leverage** ($L < 1$): Antipattern (more DOF than capabilities)

Example 2.9 (Leverage Calculations). • **SSOT:** $\text{DOF} = 1$, $\text{Cap} = \{F, \text{uses of } F\}$ where each new use adds a capability

$\Rightarrow L$ grows without bound as derivations accumulate

- **Scattered Code (n copies):** $\text{DOF} = n$, $\text{Cap} = \{F\}$
 $\Rightarrow L = 1/n$ (antipattern!)
- **Generic REST Endpoint:** $\text{DOF} = 1$, $\text{Cap} = \{\text{serve } n \text{ use cases}\}$
 $\Rightarrow L = n$
- **Specific Endpoints:** $\text{DOF} = n$, $\text{Cap} = \{\text{serve } n \text{ use cases}\}$
 $\Rightarrow L = 1$

Definition 2.10 (Architectural Dominance). Architecture A_1 *dominates* A_2 (written $A_1 \succeq A_2$) if:

1. $\text{Cap}(A_1) \supseteq \text{Cap}(A_2)$ (at least same capabilities)
2. $L(A_1) \geq L(A_2)$ (at least same leverage)

A_1 *strictly dominates* A_2 (written $A_1 \succ A_2$) if $A_1 \succeq A_2$ with at least one inequality strict.

2.5 Modification Complexity

Definition 2.11 (Modification Complexity). For requirement change δR , the *modification complexity* is:

$$M(A, \delta R) = \text{expected number of independent changes to implement } \delta R$$

Theorem 2.12 (Modification Bounded by DOF). *For all architectures A and requirement changes δR :*

$$M(A, \delta R) \leq \text{DOF}(A)$$

with equality when δR affects all components.

2

Proof. By Definition 2.3, DOF counts independent axes. A requirement change δR decomposes into sub-changes along individual axes; sub-changes along distinct axes are independent. Therefore the number of independent changes is at most the number of axes, which is $\text{DOF}(A)$. Equality holds when δR has a non-trivial projection onto every axis. ■

Example 2.13 (SSOT vs Scattered). Consider changing a structural fact F with n use sites:

- **SSOT:** $M = 1$ (change at source, derivations update automatically)
- **Scattered:** $M = n$ (must change each copy independently)

2.6 Formalization in Lean

All definitions in this section are formalized in `Leverage/Foundations.lean`:

- **Architecture:** Structure with components, state, transitions, requirements
- **Architecture.leverage:** Leverage metric
- **Architecture.dominates:** Dominance relation
- **compose_dof:** Proposition 2.4
- **modification_eq_dof:** Theorem 2.12

3 Probability Model

We derive the relationship between DOF and error probability from `AbstractClassSystem`'s axis independence theorem.

3.1 Error Independence from Axis Orthogonality

The independence of errors is not an axiom; it is a consequence of axis orthogonality proved in `AbstractClassSystem` [23].

Theorem 3.1 (Error Independence). *If axes $\{A_1, \dots, A_n\}$ are orthogonal (`AbstractClassSystem`, Theorem `minimal_complete_unique_orthogonal`), then errors along each axis are statistically independent.*

3

² : L37 ³ : L23

Proof. By Definition 2.1, the state space is the product $S = \prod_{c \in C} S_c$. **AbstractClassSystem**'s orthogonality theorem (`minimal_complete_unique_orthogonal`) establishes that a minimal complete axis set partitions the state space into independent dimensions: axis A_i is a projection onto a factor of the product, and $A_i \perp A_j$ means the projection onto factor i carries no information about factor j .

An *error along axis A_i* is the event E_i that the value on factor i deviates from specification. Since the factors are independent dimensions of the product space, the probability measure factorizes:

$$P(E_i \mid \text{state of factors } \neq i) = P(E_i).$$

This is the standard characterization of independence for product probability spaces: the marginal on factor i is independent of the marginal on any other factor. Therefore $P(E_i \cap E_j) = P(E_i) \cdot P(E_j)$ for all $i \neq j$, and by induction the n events $E_1, \dots, E_n$ are mutually independent. ■

Corollary 3.2 (DOF = Independent Error Sources). *DOF(A) = n implies n independent sources of error, each with probability p.*

4

Proof. DOF counts independent axes (Section 2, Definition 2.3). By Theorem 3.1, independent axes have independent errors. ■

Theorem 3.3 (Error Compounding). *For a system to be correct, all n independent axes must be error-free. Errors compound multiplicatively.*

5

Proof. By **Ssot**'s coherence theorem (`oracle_arbitrary`), incoherence in any axis violates system correctness. An error in axis A_i introduces incoherence along A_i . Therefore, correctness requires $\bigwedge_{i=1}^n \neg \text{error}(A_i)$. By Theorem 3.1, this probability is $(1 - p)^n$. ■

3.2 Error Probability Formula

Theorem 3.4 (Error Probability). *For architecture with n DOF and per-component error rate p:*

$$P_{\text{error}}(n) = 1 - (1 - p)^n$$

6

Proof. By Theorem 3.1 (derived from **AbstractClassSystem**'s orthogonality), each DOF has independent error probability p , so each is correct with probability $(1 - p)$. By Theorem 3.3, all n DOF must be correct:

$$P_{\text{correct}}(n) = (1 - p)^n$$

Therefore:

$$P_{\text{error}}(n) = 1 - P_{\text{correct}}(n) = 1 - (1 - p)^n$$

■

Corollary 3.5 (Linear Approximation). *For fixed $p > 0$, the linear expected-error model and the exact series model induce the same ordering on architectures by DOF.*

7

⁴ : L15, L23 ⁵ : L20, L41 ⁶ : L24, L40 ⁷ : L16, L39

Proof. By Theorem 3.10, ordering equivalence is exact under the discrete model used in the mechanization. ■

Corollary 3.6 (DOF-Error Monotonicity). *For architectures A_1, A_2 :*

$$DOF(A_1) < DOF(A_2) \implies P_{error}(A_1) < P_{error}(A_2)$$

8

Proof. $P_{error}(n) = 1 - (1 - p)^n$ is strictly increasing in n for $p \in (0, 1)$. ■

3.3 Expected Errors

Theorem 3.7 (Expected Error Bound). *Expected number of errors in architecture A :*

$$\mathbb{E}[\# \text{ errors}] = p \cdot DOF(A)$$

9

Proof. By linearity of expectation:

$$\mathbb{E}[\# \text{ errors}] = \sum_{i=1}^{DOF(A)} P(\text{error in } DOF_i) = \sum_{i=1}^{DOF(A)} p = p \cdot DOF(A)$$

■

Example 3.8 (Concrete Calculations). Assume $p = 0.01$ (1% per-component error rate):

- $DOF = 1$: $P_{error} = 1 - 0.99 = 0.01$ (1%)
- $DOF = 10$: $P_{error} = 1 - 0.99^{10} \approx 0.096$ (9.6%)
- $DOF = 100$: $P_{error} = 1 - 0.99^{100} \approx 0.634$ (63.4%)

3.4 Connection to Reliability Theory

The error model has a direct interpretation in classical reliability theory [20], connecting software architecture to a mature mathematical framework with 60+ years of theoretical development.

Theorem 3.9 (DOF-Reliability Isomorphism). *An architecture with $DOF = n$ is isomorphic to a series reliability system with n components. The isomorphism:*

1. **Preserves ordering:** *If $DOF(A_1) < DOF(A_2)$, then $P_{error}(A_1) < P_{error}(A_2)$*
2. **Is invertible:** *Round-trip mapping preserves DOF exactly*
3. **Connects domains:** *$P_{error}(n) = 1 - R_{series}(n)$ where $R_{series}(n) = (1 - p)^n$*

10

Interpretation: Each DOF is a “component” that must work correctly. This is the reliability analog of Theorem 3.1, which derives error independence from axis orthogonality.

⁸ : L33 ⁹ : L25-26 ¹⁰ : L22, L27

Theorem 3.10 (Ordering Equivalence (Exact)). *For architectures A_1, A_2 with equal capabilities and error rate $p > 0$, the linear model and the exact series model induce the same DOF ordering:*

$$DOF(A_1) \leq DOF(A_2) \iff (\mathbb{E}[\text{errors}(A_1)] \cdot d) \leq (\mathbb{E}[\text{errors}(A_2)] \cdot d),$$

where d is the denominator of the discrete error-rate representation.

11

Proof. This is theorem `ordering_equivalence_exact` in `Leverage/Probability.lean`. ■

Linear Approximation Justification: For small p (the software engineering regime where $p \approx 0.01$), the linear model $P_{\text{error}} \approx n \cdot p$ is:

1. Order-equivalent to the exact series model in the mechanized discrete representation
2. Monotone in DOF for fixed positive error rate
3. Cleanly formalized in natural-number arithmetic

3.5 Epistemic Grounding

The probability model is not axiomatic; it is derived from the epistemic foundations in `AbstractClassSystem` and `Ssot`:

1. `AbstractClassSystem` proves axis orthogonality (`minimal_complete_unique_orthogonal`)
2. **Theorem 3.1** derives error independence from orthogonality
3. `Ssot` establishes that $DOF = 1$ guarantees coherence (`oracle_arbitrary`)
4. **Theorem 3.3** connects errors to incoherence

This derivation chain ensures the probability model rests on proved foundations, not assumed axioms.

3.6 Leverage Gap: Quantitative Predictions

The leverage framework provides *quantitative*, empirically testable predictions about modification costs.

Theorem 3.11 (Leverage Gap). *For architectures with equal capabilities, the modification cost ratio equals the inverse leverage ratio:*

$$\frac{\text{ModCost}(A_2)}{\text{ModCost}(A_1)} = \frac{DOF(A_2)}{DOF(A_1)} = \frac{L(A_1)}{L(A_2)}$$

12

Theorem 3.12 (Testable Prediction). *If architecture A_1 has $n \times$ lower DOF than A_2 (for equal capabilities), then A_1 requires $n \times$ fewer expected modifications. This is empirically testable against PR/commit data.*

13

¹¹ : L32, L39 ¹² : L30 ¹³ : L42

Corollary 3.13 (DOF Ratio Predicts Error Ratio). *The ratio of expected errors between two architectures equals the ratio of their DOF:*

$$\frac{\mathbb{E}[\text{errors}(A_2)]}{\mathbb{E}[\text{errors}(A_1)]} = \frac{\text{DOF}(A_2)}{\text{DOF}(A_1)}$$

14

These theorems transform architectural intuitions into testable hypotheses. A claim that “Pattern X is 3× better than Pattern Y” can be verified by comparing DOF and measuring modification frequency in real codebases.

3.7 Formalization

Formalized in `Leverage/Probability.lean`:

- `dof_reliability_isomorphism`: Theorem 3.9 (the central isomorphism)
- `isomorphism_preserves_failure_ordering`: Ordering preservation
- `isomorphism_roundtrip`: Invertibility proof
- `ordering_equivalence_exact`: Theorem 3.10 (exact ordering equivalence)
- `linear_model_preserves_ordering`: Linear ordering support
- `leverage_gap`: Theorem 3.11
- `testable_modification_prediction`: Theorem 3.12
- `dof_ratio_predicts_error_ratio`: Corollary 3.13
- `lower_dof_lower_errors`: Corollary 3.6
- `expected_errors_from_linearity`: Theorem 3.7
- `ssot_minimal_errors`: SSOT minimality

4 Main Theorems

We prove the core software-engineering results connecting leverage to error probability and architectural optimality. The theoretical structure is:

1. **DOF-Reliability Isomorphism**: maps architecture to reliability theory
2. **Leverage-Error Tradeoff**: connects leverage to error probability
3. **Optimality Criterion**: identifies the best architecture in a fixed capability class
4. **Composition Stability**: composition preserves leverage lower bounds

¹⁴ : L21

4.1 Recap: DOF-Reliability Isomorphism

The core theoretical contribution is that DOF maps formally to series-system components. This enables the later software-engineering results by connecting architectural change structure to a mature mathematical framework for failure composition.

Key properties of the isomorphism:

- **Preserves ordering:** if $\text{DOF}(A_1) < \text{DOF}(A_2)$, then $P_{\text{error}}(A_1) < P_{\text{error}}(A_2)$
- **Invertible:** an architecture can be reconstructed from its series-system representation
- **Compositional:** $\text{DOF}(A_1 \oplus A_2) = \text{DOF}(A_1) + \text{DOF}(A_2)$

4.2 The Leverage Maximization Principle

Given the DOF-Reliability Isomorphism, the following is an actionable corollary:

Theorem 4.1 (Leverage Maximization Principle). *For any architectural decision with alternatives $A_1, \dots, A_n$ meeting capability requirements, the optimal choice maximizes leverage:*

$$A^* = \arg \max_{A_i} L(A_i) = \arg \max_{A_i} \frac{|\text{Capabilities}(A_i)|}{\text{DOF}(A_i)}$$

15

Note. This is not the deepest theorem in the formal stack; it is the software-engineering decision rule extracted from the reliability interpretation of DOF.

4.3 Leverage-Error Tradeoff

Theorem 4.2 (Leverage-Error Tradeoff). *For architectures A_1, A_2 with equal capabilities:*

$$\text{Cap}(A_1) = \text{Cap}(A_2) \wedge L(A_1) > L(A_2) \implies P_{\text{error}}(A_1) < P_{\text{error}}(A_2)$$

16

Proof. Given equal capabilities, the inequality $L(A_1) > L(A_2)$ reduces to $\text{DOF}(A_1) < \text{DOF}(A_2)$. The monotonicity result from the probability section then gives the conclusion. ■

Corollary. Maximizing leverage minimizes error probability for fixed capabilities.

4.4 Architectural Decision Criterion

Theorem 4.3 (Optimal Architecture). *Given requirements R , architecture A^* is optimal in its capability class if and only if:*

1. $\text{Cap}(A^*) \supseteq R$ (feasibility)
2. for every A' with $\text{Cap}(A') = \text{Cap}(A^*)$ and $\text{Cap}(A') \supseteq R$, one has $L(A^*) \geq L(A')$ (maximality in the fixed capability class)

17

¹⁵ : L28, L31 ¹⁶ : L29 ¹⁷ : L34, L38

Proof. If both conditions hold, then A^* is feasible and has maximal leverage in its capability class, so Theorem 4.2 gives minimal error probability in that class. Conversely, if either condition fails, then A^* is either infeasible or dominated by another architecture in the same capability class. ■

Decision Procedure:

1. Enumerate candidate architectures $\{A_1, \dots, A_n\}$
2. Filter to those with $\text{Cap}(A_i) \supseteq R$
3. Choose $A^* = \arg \max_i L(A_i)$

4.5 Leverage Composition

Theorem 4.4 (Leverage Composition). *For modular architecture $A = A_1 \oplus A_2$ with disjoint components:*

1. $\text{DOF}(A) = \text{DOF}(A_1) + \text{DOF}(A_2)$
2. if $L(A_1) \geq 1$ and $L(A_2) \geq 1$, then $L(A) \geq 1$

18

Proof. Part (1) is the DOF additivity result from the foundations section. Part (2) follows by summing the inequalities $|\text{Cap}(A_i)| \geq \text{DOF}(A_i)$ for $i = 1, 2$. ■

Remark 4.5 (Composition Tax). Composition preserves baseline leverage floors, but it does not preserve single-source structure automatically: composing two subsystems with one independent change locus each produces a system with two such loci. For software architecture, this is the consolidation tax behind modular and distributed designs.

4.6 Formalization

These results are formalized in `Leverage/Theorems.lean` and `Leverage/Probability.lean`. The TOSEM split foregrounds the software interpretation of those results rather than the thermodynamic layer developed in the companion split.

5 Instances

We demonstrate that the leverage framework unifies prior results and applies to diverse architectural decisions.

5.1 Instance 1: Single Source of Truth (SSOT)

We previously formalized the DRY principle, proving that Python uniquely provides SSOT for structural facts via definition-time hooks and introspection. Here we show SSOT is an instance of leverage maximization.

¹⁸ : L18-19

5.1.1 Prior Result

Published Theorem: A language enables SSOT for structural facts if and only if it provides (1) definition-time hooks AND (2) introspectable derivation results. Python is the only mainstream language satisfying both requirements.

Modification Complexity: For structural fact F with n use sites:

- SSOT: $M(\text{change } F) = 1$ (modify source, derivations update automatically)
- Non-SSOT: $M(\text{change } F) = n$ (modify each use site independently)

5.1.2 Leverage Perspective

Definition 5.1 (SSOT Architecture). Architecture A_{SSOT} for structural fact F has:

- Single source S defining F
- Derived use sites updated automatically from S
- $\text{DOF} = 1$ (only S is independently modifiable)

Definition 5.2 (Non-SSOT Architecture). Architecture $A_{\text{non-SSOT}}$ for structural fact F with n use sites has:

- n independent definitions (copied or manually synchronized)
- $\text{DOF} = n$ (each definition independently modifiable)

Theorem 5.3 (SSOT Leverage Dominance). *For structural fact with n use sites:*

$$\frac{L(A_{\text{SSOT}})}{L(A_{\text{non-SSOT}})} = n$$

19

Proof. Both architectures provide same capabilities: $|\text{Cap}(A_{\text{SSOT}})| = |\text{Cap}(A_{\text{non-SSOT}})| = c$.

DOF:

$$\begin{aligned} \text{DOF}(A_{\text{SSOT}}) &= 1 \\ \text{DOF}(A_{\text{non-SSOT}}) &= n \end{aligned}$$

Leverage:

$$\begin{aligned} L(A_{\text{SSOT}}) &= c/1 = c \\ L(A_{\text{non-SSOT}}) &= c/n \end{aligned}$$

Ratio:

$$\frac{L(A_{\text{SSOT}})}{L(A_{\text{non-SSOT}})} = \frac{c}{c/n} = n$$

■

Corollary 5.4 (Unbounded Advantage). *As use sites grow ($n \rightarrow \infty$), leverage advantage grows unbounded.*

¹⁹ : L9, L11

Corollary 5.5 (Error Probability). *For small p :*

$$\frac{P_{\text{error}}(A_{\text{non-SSOT}})}{P_{\text{error}}(A_{\text{SSOT}})} \approx n$$

Connection to Prior Work: Our published Theorem 6.3 (Unbounded Complexity Gap) showed $M(\text{SSOT}) = O(1)$ vs $M(\text{non-SSOT}) = \Omega(n)$. Theorem 5.3 provides the leverage perspective: SSOT achieves n -times better leverage.

5.2 Instance 2: Nominal Typing Dominance

We previously proved nominal typing strictly dominates structural and duck typing for OO systems with inheritance. Here we show this is an instance of leverage maximization.

5.2.1 Prior Result

Published Theorems:

1. Theorem 3.13 (Provenance Impossibility): No shape discipline can compute provenance
2. Theorem 3.19 (Capability Gap): Gap = B-dependent queries = {provenance, identity, enumeration, conflict resolution}
3. Theorem 3.5 (Strict Dominance): Nominal strictly dominates duck typing

5.2.2 Leverage Perspective

Definition 5.6 (Typing Discipline as Architecture). A typing discipline D is an architecture where:

- Components = type checker, runtime dispatch, introspection APIs
- Capabilities = queries answerable by the discipline

Duck Typing: Uses only Shape axis (S : methods, attributes)

- Capabilities: Shape checking (“Does object have method m ?”)
- Cannot answer: provenance, identity, enumeration, conflict resolution

Nominal Typing: Uses Name + Bases + Shape axes ($N + B + S$)

- Capabilities: All duck capabilities PLUS 4 B-dependent capabilities
- Can answer: “Which type provided method m ?” (provenance), “Is this exactly type T ?” (identity), “List all subtypes of T ” (enumeration), “Which method wins in diamond?” (conflict)

Theorem 5.7 (Nominal Leverage Dominance). *Assuming $\text{DOF}(\text{Nominal}) = \text{DOF}(\text{Duck}) = d$ (equal implementation cost hypothesis):*

$$L(\text{Nominal}) > L(\text{Duck})$$

20

²⁰ : L12-13

Proof. Let $c_{\text{duck}} = |\text{Cap}(\text{Duck})|$ and $c_{\text{nominal}} = |\text{Cap}(\text{Nominal})|$.

By Theorem 3.19 (published): $c_{\text{nominal}} = c_{\text{duck}} + 4$.

By hypothesis: $\text{DOF}(\text{Nominal}) = \text{DOF}(\text{Duck}) = d$.

Therefore:

$$L(\text{Nominal}) = \frac{c_{\text{duck}} + 4}{d} > \frac{c_{\text{duck}}}{d} = L(\text{Duck})$$

Status of hypothesis. The equal-DOF assumption is formalized as an explicit premise in the Lean mechanization (L13). Whether it holds in practice depends on the implementation. Abstractly: the typing discipline is one component; DOF is the number of independent change points in that component. The hypothesis says adding name-based dispatch (N + B axes) does not introduce more independent change points than shape-based dispatch (S axis alone). ■

Connection to Prior Work: Our published Theorem 3.5 (Strict Dominance) showed nominal typing provides strictly more capabilities for same DOF cost. Theorem 5.7 provides the leverage formulation.

5.3 Instance 3: Microservices Architecture

Should a system use microservices or a monolith? How many services are optimal? The leverage framework provides answers. This architectural style, popularized by Fowler and Lewis [10], traces its roots to the Unix philosophy of “doing one thing well” [21].

5.3.1 Architecture Comparison

Monolith:

- Components: Single deployment unit
- DOF = 1
- Capabilities: Basic functionality, simple deployment

n **Microservices:**

- Components: n independent services
- DOF = n (each service independently deployable/modifiable)
- Additional Capabilities: Independent scaling, independent deployment, fault isolation, team autonomy, polyglot persistence [10]

5.3.2 Leverage Analysis

Let c_0 = capabilities provided by monolith.

Let $\Delta c = 5$ denote the additional capabilities from microservices: independent scaling, independent deployment, fault isolation, team autonomy, and polyglot persistence.

Leverage:

$$\begin{aligned} L(\text{Monolith}) &= c_0/1 = c_0 \\ L(n \text{ Microservices}) &= (c_0 + \Delta c)/n = (c_0 + 5)/n \end{aligned}$$

Break-even Point:

$$L(\text{Microservices}) \geq L(\text{Monolith}) \iff \frac{c_0 + 5}{n} \geq c_0 \iff n \leq 1 + \frac{5}{c_0}$$

Interpretation: If base capabilities $c_0 = 5$, then $n \leq 2$ services is optimal. For $c_0 = 20$, up to $n = 1.25$ (i.e., monolith still better). Microservices justified only when additional capabilities significantly outweigh DOF cost.

5.4 Instance 4: REST API Design

Generic endpoints vs specific endpoints: a leverage tradeoff.

5.4.1 Architecture Comparison

Specific Endpoints: One endpoint per use case

- Example: GET /users, GET /posts, GET /comments, ...
- For n use cases: DOF = n
- Capabilities: Serve n use cases

Generic Endpoint: Single parameterized endpoint

- Example: GET /resources/:type/:id
- DOF = 1
- Capabilities: Serve n use cases (same as specific)

5.4.2 Leverage Analysis

$$L(\text{Generic}) = n/1 = n$$

$$L(\text{Specific}) = n/n = 1$$

Advantage: $L(\text{Generic})/L(\text{Specific}) = n$

Tradeoff: Generic endpoint has higher leverage but may sacrifice:

- Type safety (dynamic routing)
- Specific validation per resource
- Tailored response formats

Decision Rule: Use generic if $n > k$ where k is complexity threshold (typically $k \approx 3-5$).

5.5 Instance 5: Configuration Systems

Convention over configuration (CoC) reduces developer decisions and preserves flexibility [14]. In this framework it is leverage maximization via defaults.

5.5.1 Architecture Comparison

Explicit Configuration: Must set all m parameters

- DOF = m (each parameter independently set)
- Capabilities: Configure m aspects

Convention over Configuration: Provide defaults, override only k parameters

- DOF = k where $k \ll m$
- Capabilities: Configure same m aspects (defaults handle rest)

Example (Rails vs Java EE):

- Rails: 5 config parameters (convention for rest)
- Java EE: 50 config parameters (explicit for all)

5.5.2 Leverage Analysis

$$L(\text{Convention}) = m/k$$

$$L(\text{Explicit}) = m/m = 1$$

Advantage: $L(\text{Convention})/L(\text{Explicit}) = m/k$

For Rails example: $m/k = 50/5 = 10$ (10× leverage improvement).

5.6 Instance 6: Database Schema Normalization

Normalization eliminates redundancy, maximizing leverage. This concept, introduced by Codd [7], is the foundation of relational database design [9].

5.6.1 Architecture Comparison

Consider customer address stored in database:

Denormalized (Address in 3 tables):

- **Users** table: address columns
- **Orders** table: shipping address columns
- **Invoices** table: billing address columns
- DOF = 3 (address stored 3 times)

Normalized (Address in 1 table):

- **Addresses** table: single source
- Foreign keys from **Users**, **Orders**, **Invoices**
- DOF = 1 (address stored once)

5.6.2 Leverage Analysis

Both provide same capability: store/retrieve addresses.

$$L(\text{Normalized}) = c/1 = c$$

$$L(\text{Denormalized}) = c/3$$

Advantage: $L(\text{Normalized})/L(\text{Denormalized}) = 3$

Modification Complexity:

- Change address format: Normalized $M = 1$, Denormalized $M = 3$
- Error probability: $P_{\text{denorm}} = 3p$ vs $P_{\text{norm}} = p$

Tradeoff: Normalization increases leverage but may sacrifice query performance (joins required).

5.7 Summary of Instances

Instance	High Leverage	Low Leverage	Ratio
SSOT	DOF = 1	DOF = n	n
Nominal Typing	$c + 4$ caps, DOF d	c caps, DOF d	$(c + 4)/c$
Microservices	Monolith (DOF = 1)	n services (DOF = n)	$n/(c_0 + 5)$
REST API	Generic (DOF = 1)	Specific (DOF = n)	n
Configuration	Convention (DOF = k)	Explicit (DOF = m)	m/k
Database	Normalized (DOF = 1)	Denormalized (DOF = n)	n

Pattern: High leverage architectures achieve n -fold improvement where n is the consolidation factor (use sites, services, endpoints, parameters, or redundant storage).

6 Practical Demonstration

We demonstrate the leverage framework by showing how DOF collapse patterns manifest in OpenHCS, a production 45K LoC Python bioimage analysis platform. This section presents qualitative before/after examples illustrating the leverage archetypes, with PR #44 providing a publicly verifiable anchor.

6.1 The Leverage Mechanism

For a before/after pair $A_{\text{pre}}, A_{\text{post}}$, the **structural leverage factor** is:

$$\rho := \frac{\text{DOF}(A_{\text{pre}})}{\text{DOF}(A_{\text{post}})}.$$

If capabilities are preserved, leverage scales exactly by ρ . The key insight: when $\text{DOF}(A_{\text{post}}) = 1$, we achieve $\rho = n$ where n is the original DOF count.

What counts as a DOF? Independent *definition loci*: manual registration sites, independent override parameters, separately defined endpoints/handlers/rules, duplicated schema/format definitions. The unit is “how many places can drift apart,” not lines of code.

6.2 Verifiable Example: PR #44 (Contract Enforcement)

PR #44 (“UI Anti-Duck-Typing Refactor”) in the OpenHCS repository provides a publicly verifiable demonstration of DOF collapse:

Before (duck typing): The `ParameterFormManager` class used scattered `hasattr()` checks throughout the codebase. Each dispatch point was an independent DOF: a location that could drift, contain typos, or miss updates when widget interfaces changed.

After (nominal ABC): A single `AbstractFormWidget` ABC defines the contract. All dispatch points collapsed to one definition site. The ABC provides fail-loud validation at class definition time rather than fail-silent behavior at runtime.

Leverage interpretation: DOF collapsed from n scattered dispatch points to 1 centralized ABC. By Theorem 3.1, this achieves $\rho = n$ leverage improvement. The specific value of n is verifiable by inspecting the PR diff.

6.3 Leverage Archetypes

The framework identifies recurring patterns where DOF collapse occurs:

6.3.1 Archetype 1: SSOT (Single Source of Truth)

Pattern: Scattered definitions $\rightarrow$ single authoritative source.

Mechanism: Metaclass auto-registration, decorator-based derivation, or introspection-driven generation eliminates manual synchronization.

Before: Define class + register in dispatch table (2 loci per type). **After:** Define class; metaclass auto-registers (1 locus per type).

Leverage: $\rho = 2$ per type; compounds across n types.

6.3.2 Archetype 2: Convention over Configuration

Pattern: Explicit parameters $\rightarrow$ sensible defaults with override.

Mechanism: Framework provides defaults; users override only non-standard values.

Before: Specify all m configuration parameters explicitly. **After:** Override only $k \ll m$ parameters; defaults handle rest.

Leverage: $\rho = m/k$.

6.3.3 Archetype 3: Generic Abstraction

Pattern: Specific implementations $\rightarrow$ parameterized generic.

Mechanism: Factor common structure into generic endpoint/handler with parameters for variation.

Before: n specific endpoints with duplicated logic. **After:** 1 generic endpoint with n parameter instantiations.

Leverage: $\rho = n$.

6.3.4 Archetype 4: Centralization

Pattern: Scattered cross-cutting concerns $\rightarrow$ centralized handler.

Mechanism: Middleware, decorators, or aspect-oriented patterns consolidate error handling, logging, authentication, etc.

Before: Each call site handles concern independently. **After:** Central handler; call sites delegate.

Leverage: $\rho = n$ where n is number of call sites.

6.4 Summary

The leverage framework identifies a common mechanism across diverse refactoring patterns: DOF collapse yields proportional leverage improvement. Whether the pattern is SSOT, convention-over-configuration, generic abstraction, or centralization, the mathematical structure is identical: reduce DOF while preserving capabilities.

PR #44 provides a verifiable anchor demonstrating this mechanism in practice. The qualitative value lies not in aggregate statistics but in the *mechanism*: once understood, the pattern applies wherever scattered definitions can be consolidated.

7 Related Work

7.1 Software Architecture Metrics

Coupling and Cohesion [24]: Introduced coupling (inter-module dependencies) and cohesion (intra-module relatedness). Recommend high cohesion, low coupling.

Difference: Our framework is capability-aware. High cohesion correlates with high leverage (focused capabilities per module), but we formalize the connection to error probability.

Cyclomatic Complexity [19]: Counts decision points in code. Higher values are commonly used as a risk indicator, though empirical studies on defect correlation show mixed results.

Difference: Complexity measures local control flow; leverage measures global architectural DOF. Orthogonal concerns.

7.2 Design Patterns

Gang of Four [11]: Catalogued 23 design patterns (Singleton, Factory, Observer, etc.). Patterns codify best practices but lack formal justification.

Connection: Many patterns maximize leverage:

- **Factory Pattern:** Centralizes object creation (DOF = 1 for creation logic)
- **Strategy Pattern:** Encapsulates algorithms (DOF = 1 per strategy family)
- **Template Method:** Defines algorithm skeleton (DOF = 1 for structure)

Our framework explains *why* these patterns work: they maximize leverage.

7.3 Technical Debt

Cunningham [8]: Introduced technical debt metaphor. Poor design creates “debt” that must be “repaid” later.

Connection: Low leverage = high technical debt. Scattered DOF (non-SSOT, denormalized schemas, specific endpoints) create debt. High leverage architectures minimize debt.

7.4 Formal Methods in Software Architecture

Architecture Description Languages (ADLs): Wright [1], ACME [13], Aesop [12]. Formalize architecture structure but not decision-making. See also Shaw and Garlan [22].

Difference: ADLs describe architectures; our framework prescribes optimal architectures via leverage maximization.

ATAM and CBAM: Architecture Tradeoff Analysis Method [15] and Cost Benefit Analysis Method [2]. Evaluate architectures against quality attributes (performance, modifiability, security). See also Bass et al. [4].

Difference: ATAM is qualitative; our framework provides quantitative optimization criterion (maximize L).

Necessity Specifications: Mackay et al. [17] formalize *necessity specifications*: robustness guarantees that modules do only what their specification requires, even under adversarial clients. Soundness is mechanized in Coq.

Connection: Necessity specifications address *behavioral minimality*: modules commit to no more behavior than required. Our framework addresses *structural minimality*: architectures commit to no more DOF than required. Both derive minimal commitments from requirements and prove sufficiency.

7.5 Software Metrics Research

Chidamber-Kemerer Metrics [6]: Object-oriented metrics (WMC, DIT, NOC, CBO, RFC, LCOM). Empirical validation studies [3] found these metrics correlate with external quality attributes.

Connection: Metrics like CBO (Coupling Between Objects) and LCOM (Lack of Cohesion) correlate with DOF. High CBO $\implies$ high DOF. Our framework provides theoretical foundation.

7.6 Metaprogramming and Reflection

Reflection [18]: Languages with reflection enable introspection and intercession. Essential for metaprogramming.

Connection: Reflection enables high leverage (SSOT). Our prior work showed Python’s definition-time hooks + introspection uniquely enable SSOT for structural facts.

Metaclasses [5, 16]: Early metaobject and metaclass machinery formalized in CommonLoops; the Metaobject Protocol codified in Kiczales et al.’s AMOP. Enable metaprogramming patterns.

Application: Metaclasses are high-leverage mechanism (DOF = 1 for class structure, unlimited derivations).

8 Extension: Weighted Leverage

The basic leverage framework treats all errors equally. In practice, different decisions carry different consequences. This section extends our framework with *weighted leverage* to capture heterogeneous error severity.

8.1 Weighted Decision Framework

Definition 8.1 (Weighted Decision). A **weighted decision** extends an architecture with:

- **Importance weight** $w \in \mathbb{N}^+$: the relative severity of errors in this decision

- **Risk-adjusted DOF:** $\text{DOF}_w = \text{DOF} \times w$

The key insight is that a decision with importance weight w carries w times the error consequence of a unit-weight decision. This leads to:

Definition 8.2 (Weighted Leverage).

$$L_w = \frac{\text{Capabilities} \times w}{\text{DOF}_w} = \frac{\text{Capabilities}}{\text{DOF}}$$

The cancellation is intentional: weighted leverage preserves comparison properties while enabling risk-adjusted optimization.

8.2 Key Theorems

Theorem 8.3 (Weighted Pareto Optimality). *For any weighted decision d with $\text{DOF} = 1$: d is Pareto-optimal (not dominated by any alternative with higher weighted leverage).*

Proof. Suppose d has $\text{DOF} = 1$. For any d' to dominate d , we would need $d'.\text{DOF} < 1$. But $\text{DOF} \geq 1$ by definition, so no such d' exists. ■

Theorem 8.4 (Weighted Leverage Transitivity). $\forall a, b, c$: *if a has higher weighted leverage than b , and b has higher weighted leverage than c , then a has higher weighted leverage than c .*

Proof. By algebraic manipulation of cross-multiplication inequalities. Formally verified in Lean (38-line proof). ■

8.3 Practical Application: Feature Flags

Consider two approaches to feature toggle implementation:

Low Leverage (Scattered Conditionals):

- DOF: One per feature $\times$ one per use site ($n \times m$)
- Risk: Inconsistent behavior if any site is missed
- Weight: High (user-facing inconsistency)

High Leverage (Centralized Configuration):

- DOF: One per feature
- Risk: Single source of truth eliminates inconsistency
- Weight: Same importance, but $m \times$ fewer DOF

Weighted leverage ratio: $L_{\text{centralized}}/L_{\text{scattered}} = m$, the number of use sites.

8.4 Connection to Main Theorems

The weighted framework preserves all results from Sections 3–5:

- **Theorem 3.1 (Leverage-Error Tradeoff):** Holds with weighted errors
- **Theorem 3.2 (Metaprogramming Dominance):** Weight amplifies the advantage
- **Theorem 3.4 (Optimality):** Weighted optimization finds risk-adjusted optima
- **SSOT Dominance:** Weight w makes $n \times w$ leverage advantage

All proofs verified in Lean: `Leverage/WeightedLeverage.lean` (348 lines, 0 sorry placeholders).

9 Conclusion

This paper develops leverage as a software-architecture metric grounded in structural change points rather than stylistic preference. The central software result is that, in fixed capability classes, lower DOF yields lower error probability and lower modification burden, so higher leverage becomes a principled selection criterion rather than an aesthetic slogan.

The framework explains a wide range of familiar design advice in a single structural language. Single source of truth, nominal contracts, generic abstractions, convention over configuration, and normalization all realize the same mechanism: consolidate independently drifting definition loci while preserving capabilities. The OpenHCS example illustrates how that mechanism appears in practice through explicit DOF collapse.

The present TOSEM-oriented split isolates the software-engineering contribution from the mathematical-physics material developed in the companion version of paper 3. The next step for the software paper is empirical strengthening: broader case studies, a reproducible DOF measurement protocol, and a fuller evaluation of threats to validity.

10 Lean Proof Artifacts

This appendix reports machine-check status and proof traceability directly from source and generated mapping artifacts.

10.1 Verification Status

Lean summary: 30495 lines, 1478 theorems/lemmas, 0 sorry, across 104 files.

File	Lines	Theorems/Lemmas
<code>Leverage.lean</code>	115	0
<code>Leverage/Foundations.lean</code>	234	13
<code>Leverage/Probability.lean</code>	841	39
<code>Leverage/Theorems.lean</code>	303	20
<code>Leverage/Examples.lean</code>	184	4
<code>Leverage/WeightedLeverage.lean</code>	348	13
<code>lakefile.lean</code>	25	0
Total	30495	1478

Build command: `cd proofs && lake build`

10.2 Claim Coverage Matrix

Paper claim	Lean handle
Corollary 3.2: DOF = Independent Error Sources	L15 , L23
Corollary 3.6: DOF-Error Monotonicity	L33
Corollary 3.13: DOF Ratio Predicts Error Ratio	L21 , L42
Corollary 3.5: Linear Approximation	L16 , L39
Definition 2.4: Architecture	L17 , L19
Theorem 3.10: Ordering Equivalence (Exact)	L32 , L39
Theorem 4.4: Leverage Composition	L18 , L19
Theorem 3.9: DOF-Reliability Isomorphism	L22 , L27
Theorem 3.3: Error Compounding	L20 , L41
Theorem 3.1: Error Independence	L23
Theorem 3.4: Error Probability	L24 , L40
Theorem 3.7: Expected Error Bound	L25 , L26
Theorem 4.2: Leverage-Error Tradeoff	L29
Theorem 3.11: Leverage Gap	L30
Theorem 4.1: Leverage Maximization Principle	L28 , L31
Definition 2.12: Capability Set	L37
Corollary 5.7: Unbounded Advantage	L12 , L13
Theorem 4.3: Optimal Architecture	L34 , L38
Definition 5.3: SSOT Architecture	L9 , L11
Theorem 3.12: Testable Prediction	L30 , L42

Auto summary: mapped 20/20 (full=20, derived=0, unmapped=0).

10.3 Lean Handle Map

ID	Lean Handle / Source	ID	Lean Handle / Source
L9	ACS1 paper1/HandleAliases.lean	L10	ACS2 paper1/HandleAliases.lean
L11	ACS3 paper1/HandleAliases.lean	L12	ACS4 paper1/HandleAliases.lean
L13	ACS5 paper1/HandleAliases.lean	L14	ACS6 paper1/HandleAliases.lean
L15	ACS7 paper1/HandleAliases.lean	L16	ACS8 paper1/HandleAliases.lean
L17	ACS9 paper1/HandleAliases.lean	L18	ADV1 paper1/HandleAliases.lean
L19	ADV2 paper2/Ssot/HandleAliases.lean	L20	ADV3 paper2/Ssot/HandleAliases.lean
L21	ADV4 paper2/Ssot/HandleAliases.lean	L22	ADV5 paper2/Ssot/HandleAliases.lean

(continued...)

ID	Lean Handle / Source	ID	Lean Handle / Source
L23	AFM1 paper2/Ssot/HandleAliases.lean	L24	AFM10 paper2/Ssot/HandleAliases.lean
L25	AFM11 paper2/Ssot/HandleAliases.lean	L26	AFM12 paper2/Ssot/HandleAliases.lean
L27	AFM13 paper2/Ssot/HandleAliases.lean	L28	AFM14 paper2/Ssot/HandleAliases.lean
L29	AFM15 paper2/Ssot/HandleAliases.lean	L30	AFM16 paper2/Ssot/HandleAliases.lean
L31	AFM17 paper2/Ssot/HandleAliases.lean	L32	AFM18 paper2/Ssot/HandleAliases.lean
L33	AFM19 paper2/Ssot/HandleAliases.lean	L34	AFM2 paper2/Ssot/HandleAliases.lean
L37	AFM5 paper2/Ssot/HandleAliases.lean	L38	AFM6 paper2/Ssot/HandleAliases.lean
L39	AFM7 paper2/Ssot/HandleAliases.lean	L40	AFM8 paper2/Ssot/HandleAliases.lean
L41	AFM9 paper2/Ssot/HandleAliases.lean	L42	ALT1 paper1/HandleAliases.lean

10.4 Proof Hardness Index

Paper handle	Hardness profile	Regime tags	Lean support
cor:dof-errors	unspecified	-	L15, L23
cor:dof-monotone	unspecified	-	L33
cor:dof-ratio	unspecified	-	L21, L42
cor:linear-approx	unspecified	-	L16, L39
prop:dof-additive	unspecified	-	L17, L19
thm:approx-bound	unspecified	-	L32, L39
thm:composition	unspecified	-	L18, L19
thm:dof-reliability	unspecified	-	L22, L27
thm:error-compound	unspecified	-	L20, L41
thm:	unspecified	-	L23
error-independence			
thm:error-prob	unspecified	-	L24, L40
thm:expected-errors	unspecified	-	L25, L26
thm:leverage-error	unspecified	-	L29
thm:leverage-gap	unspecified	-	L30
thm:leverage-max	unspecified	-	L28, L31
thm:mod-bound	unspecified	-	L37
thm:nominal-leverage	unspecified	-	L12, L13
thm:optimal	unspecified	-	L34, L38
thm:ssot-leverage	unspecified	-	L9, L11
thm:	unspecified	-	L30, L42
testable-prediction			

Auto summary: indexed 20 claims by hardness profile (unspecified=20).

10.5 Assumption Ledger

10.6 Assumption Ledger (Auto)

Source files.

- Leverage/Examples.lean
- Leverage/Foundations.lean
- Leverage/Probability.lean

- `Leverage/Theorems.lean`
- `Leverage/WeightedLeverage.lean`

Assumption bundles and fields.

- (No assumption bundles parsed.)

Conditional closure handles.

- (No conditional theorem handles parsed.)

First-principles Bayes derivation handles.

- (No Bayes-derivation theorem handles found.)

Compact Bayes handle IDs.

- (No compact Bayes alias IDs found.)

References

- [1] Robert Allen and David Garlan. A formal basis for architectural connection. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 6(3):213–249, 1997.
- [2] Felix Bachmann, Len Bass, and Constance Heitmeyer. Characterizing, planning, and managing technical debt. Technical report, Software Engineering Institute, 2000.
- [3] Victor R. Basili, Lionel C. Briand, and Walcélio L. Melo. A validation of object-oriented design metrics as quality indicators. *IEEE Transactions on Software Engineering*, 22(10):751–761, 1996.
- [4] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 2012.
- [5] Daniel G. Bobrow, Kenneth Kahn, Gregor Kiczales, Larry Masinter, Mark Stefik, and Frank Zdybel. Commonloops: Merging lisp and object-oriented programming. *ACM SIGPLAN Notices*, 21(11):17–29, 1986.
- [6] Shyam R. Chidamber and Chris F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493, 1994.
- [7] Edgar F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.
- [8] Ward Cunningham. The wycash portfolio management system. In *OOPSLA '92 Addendum to the Proceedings*, pages 29–30. ACM, 1992.
- [9] Christopher J. Date. *An Introduction to Database Systems*. Pearson Education, 2003.
- [10] Martin Fowler and James Lewis. Microservices: a definition of this new architectural term. <https://martinfowler.com/articles/microservices.html>, 2014.

- [11] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.
- [12] David Garlan, Robert Allen, and John Ockerbloom. Exploiting style in architectural design environments. In *Proceedings of the 2nd ACM SIGSOFT Symposium on Foundations of Software Engineering*, pages 175–188, 1994.
- [13] David Garlan, Robert T. Monroe, and David Wile. Acme: An architecture description interchange language. In *Proceedings of CASCION '97*, pages 169–183, 1997.
- [14] David Heinemeier Hansson. The rails doctrine. <https://rubyonrails.org/doctrine>, 2005.
- [15] Rick Kazman, Mark Klein, and Paul Clements. Atam: Method for architecture evaluation. Technical Report CMU/SEI-2000-TR-004, Carnegie Mellon University, Software Engineering Institute, 2000.
- [16] Gregor Kiczales, Jim des Rivières, and Daniel G. Bobrow. *The Art of the Metaobject Protocol*. MIT Press, 1991.
- [17] Julian Mackay, Sophia Drossopoulou, James Noble, and Susan Eisenbach. Necessity specifications for robustness. Zenodo, 2022. Coq mechanization of necessity specifications for module robustness.
- [18] Pattie Maes. Concepts and experiments in computational reflection. *ACM SIGPLAN Notices*, 22(12):147–155, 1987.
- [19] Thomas J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4):308–320, 1976.
- [20] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, 2013. Discusses RAID and reliability formulas.
- [21] Rob Pike and Brian W. Kernighan. Program design in the unix environment. *AT&T Bell Laboratories Technical Journal*, 63(8):1595–1605, 1984.
- [22] Mary Shaw and David Garlan. *Software Architecture: Perspectives on an Emerging Discipline*. Prentice Hall, 1996.
- [23] Tristan Simas. Identification capacity and rate-query tradeoffs in classification systems. Zenodo, 2025. Paper 1 of this series. Proves axis orthogonality theorem.
- [24] Wayne P. Stevens, Glenford J. Myers, and Larry L. Constantine. Structured design. *IBM Systems Journal*, 13(2):115–139, 1974.